



Introducción a las Finanzas y a la Empresa

Eduardo Selim Martínez Mayorga

2026-2

Licenciatura en
Ciencia de Datos



Sesión: 21 de abril de 2026

Introducción a las Metodologías Ágiles

- Metodologías ágiles → Enfoques iterativos e incrementales para el desarrollo de proyectos

- Enfatizan en
 - flexibilidad
 - colaboración
 - entrega continua de valor

características
específicas prácticas

filosofía del framework
Necesito saber todas
las características de
un software p/ mejorar
a desarrollarlo

- Surgieron como respuesta a las limitaciones de los métodos tradicionales en cascada, priorizando la adaptación al cambio sobre el seguimiento rígido de planes.

- Principios fundamentales:

- Individuos e interacciones sobre procesos y herramientas
- Software funcionando sobre documentación extensiva
- Colaboración con el cliente sobre negociación contractual
- Responder ante el cambio sobre seguir un plan

“Tiene que ser,
no tiene que
ser perfecto...
aún”

El Manifiesto Agile

1
nada está
escrito en piedra.

- Creado en 2001 por 17 desarrolladores de software
 - Estableció los valores y principios que guían todas las metodologías ágiles.
 - Este documento es una referencia actual sobre la forma en que se dirigen proyectos tecnológicos
 - Promueve ciclos cortos de desarrollo, feedback constante y equipos auto-organizados.
-
- Tiene 12 principios clave. Algunos de éstos:
 - Satisfacción del cliente mediante entrega temprana y continua
 - Aceptar cambios en los requisitos, incluso en etapas tardías
 - Entrega frecuente de software funcional
 - Colaboración diaria entre negocio y desarrolladores

↳ gita: DS!

Frameworks agile populares

más popular
y el más usado.

- Existen varios frameworks que implementan los principios agile
- **SCRUM:** Framework estructurado con roles, eventos y artefactos definidos. Ideal para equipos de 3-9 personas con sprints de 1-4 semanas.
- **Kanban:** Sistema visual de administración del flujo de trabajo que limita el trabajo en progreso y maximiza la eficiencia.
- **XP (Extreme Programming):** Enfocado en prácticas técnicas de ingeniería como pair programming y TDD (Test Driven Development).
- **Lean:** Orientado a eliminar desperdicios y maximizar el valor entregado al cliente.

Agile para DS

no sabemos si
un modelo va funcionar
como queremos

- Los proyectos de Data Science tienen algunas características particulares
 - Alta incertidumbre
 - Exploración iterativa de datos
 - Resultados que dependen de la calidad de los datos y modelos
- Las metodologías *agile* se usan porque:
 - Permiten experimentación rápida con diferentes enfoques
 - Facilitan la validación temprana de hipótesis con stakeholders
 - Adaptan el alcance según los descubrimientos en los datos
 - Fomentan la colaboración entre científicos de datos y negocio
 - Reducen el riesgo de invertir meses en modelos que no agregan valor

DS con planeación rígida (cascada)

→ Todo ya
(pre)definido
y secuencial.

Los métodos en cascada fallan en DS pues:

- **Incertidumbre inherente:** No sabemos qué patrones encontraremos en los datos hasta explorarlos. Los requisitos no pueden definirse completamente al inicio.
- **Naturaleza exploratoria:** El 70-80% del tiempo se invierte en limpieza y exploración de datos, actividades difíciles de planear con precisión.
- **Validación iterativa:** Los modelos requieren varios ciclos de entrenamiento, evaluación y ajuste.
- **Dependencias de datos:** La calidad y disponibilidad de datos pueden cambiar el enfoque del proyecto completamente.

¿Qué es SCRUM?



no tiene
na
traducción
al español.

Un **scrummage** o simplemente como **scrum** es un método para reiniciar el juego en el rugby que consiste en que los jugadores se agrupan estrechamente con la cabeza hacia abajo e intentan ganar la posesión del balón

Introducción a SCRUM

- SCRUM es el framework ágil más popular
 - Utilizado por aprox 70% de los equipos ágiles globalmente
 - Fue desarrollado por Ken Schwaber y Jeff Sutherland en los años 1990s
 - Se basa en la teoría de control de procesos empíricos.
 - Características principales:
 - Equipos autoorganizados y multifuncionales
 - Trabajo dividido en *sprints* de duración fija
 - Inspección y adaptación continuas
 - Transparencia en todo el proceso
 - Framework ligero pero con reglas claras
- 2 semanas

Los 3 pilares de SCRUM

SCRUM se fundamenta en tres pilares:

1. **Transparencia:** Todos los aspectos del proceso deben ser visibles para quienes son responsables del resultado. Se utiliza un lenguaje común y una "Definición de Hecho" compartida
2. **Inspección:** Los artefactos y el progreso hacia los objetivos deben inspeccionarse frecuentemente para detectar variaciones no deseadas
3. **Adaptación:** Si en una revisión se determina que uno o más aspectos del proceso se desvían fuera de límites aceptables, el proceso debe ajustarse lo antes posible

Roles en SCRUM - Product Owner

Lista de tareas.

- El **Product Owner** es el responsable de maximizar el valor del producto resultante del trabajo del **Equipo de Desarrollo**.
- Responsabilidades principales:
 - Administrar y priorizar el Product Backlog
 - Expresar claramente los elementos del Product Backlog
 - Asegurar que el equipo entienda los elementos al nivel necesario
 - Tomar decisiones sobre qué construir y en qué orden
 - Representar las necesidades de los stakeholders

En DS suele definirse el problema de negocio, establecerse métricas de éxito, y validar que los modelos agreguen valor real

Roles en SCRUM - Scrum Master

- El Scrum Master es responsable de promover y apoyar SCRUM, ayudando a todos a entender la teoría, prácticas, reglas y valores de SCRUM
- Responsabilidades principales:
 - Facilitar eventos de SCRUM según se requiera
 - Eliminar impedimentos para el progreso del equipo
 - Entrenar al equipo en auto-organización y multifuncionalidad
 - Proteger al equipo de interrupciones externas
 - Facilitar la colaboración entre roles
- En DS ayuda a adaptar SCRUM al contexto científico, administrar dependencias técnicas y facilitar la comunicación entre áreas

Roles en SCRUM - Equipo de desarrollo

- El **equipo de desarrollo** consiste en profesionales que realizan el trabajo de entregar un Incremento potencialmente liberable del producto al final de cada *sprint*
- Características:
 - Auto-organizados: nadie les dice cómo convertir el Backlog en incrementos
 - Multifuncionales: tienen todas las habilidades necesarias
 - Sin títulos individuales, todos son "desarrolladores"
 - Sin sub-equipos (testing, arquitectura, etc.)
 - Tamaño óptimo: 3-9 personas
- En DS incluye científicxs de datos, ingenierxs de ML, analistas, y data engineers trabajando colaborativamente

El concepto de *sprint* en SCRUM

- El sprint es un contenedor de tiempo de un mes o menos durante el que se crea un Incremento de producto "hecho", utilizable y potencialmente liberable.
- Características del sprint:
 - Duración fija (1-4 semanas, típicamente 2 semanas)
 - Nuevo sprint comienza inmediatamente después del anterior
 - El alcance puede aclararse y renegociarse con el Product Owner
 - No se hacen cambios que pongan en peligro el objetivo del sprint
 - Los estándares de calidad no disminuyen
- En DS permite experimentación controlada y validación incremental de hipótesis.

Product backlog - Lista priorizada de trabajo

- El **product backlog** es una lista ordenada de todo lo que podría ser necesario en el producto y es la única fuente de requisitos para cualquier cambio a realizarse en el producto.
- Características:
 - Nunca está completo, evoluciona constantemente
 - Ordenado por valor, riesgo, prioridad y necesidad
 - Los elementos de mayor orden están más refinados
 - El Product Owner es responsable del contenido, disponibilidad y ordenamiento
- En DS incluye hipótesis a validar, experimentos, mejoras de modelos, análisis exploratorios, implementación de pipelines, y trabajo de infraestructura.

Sprint backlog

- El **sprint backlog** es el conjunto de elementos del product backlog seleccionados para el sprint, más un plan para entregar el incremento y conseguir el objetivo del sprint.
- Componentes:
 - Elementos del product backlog seleccionados
 - Plan de implementación (tareas)
 - Objetivo del sprint
- Es una predicción del equipo sobre qué funcionalidad estará en el próximo incremento
- Es propiedad del equipo de desarrollo
- Monitoreo: El progreso hacia el objetivo del sprint se rastrea diariamente, típicamente usando un burndown chart.

Sprint planning

- El **sprint planning** inicia el sprint estableciendo el trabajo a realizar.
- Es un evento time-boxed de máximo 8 horas para un sprint de un mes
- Las 2 preguntas más importantes:
- **¿Qué puede entregarse en el incremento resultante del Sprint?**
 - El equipo de desarrollo pronostica la funcionalidad a desarrollar
 - Se define el objetivo del sprint
- **¿Cómo se logrará el trabajo necesario?**
 - El equipo de desarrollo descompone elementos en tareas de un día o menos
 - Se crea el sprint backlog
- En DS se seleccionan experimentos, análisis o componentes de modelo a desarrollar y validar

Daily SCRUM - Sincronización diaria

- El **daily SCRUM** es un evento de 15 minutos para el equipo de desarrollo que se realiza cada día del sprint.
- Busca sincronizar actividades y crear un plan para las próximas 24 horas.
- Estructura típica (tres preguntas):
 - ¿Qué hice ayer que ayudó al equipo a cumplir el objetivo del sprint?
 - ¿Qué haré hoy para ayudar al equipo a cumplir el objetivo del sprint?
 - ¿Veo algún impedimento que evite que el equipo o yo cumplamos el objetivo?
- **Beneficios:** Mejora la comunicación, elimina otras reuniones, identifica impedimentos, promueve la toma rápida de decisiones y mejora el conocimiento del equipo.

Introducción a MLOps

- MLOps (*Machine Learning Operations*) es un conjunto de prácticas que combina **Machine Learning**, **DevOps** y **Data Engineering** para implementar, monitorear y mantener modelos de ML en producción de manera confiable y eficiente.
- Objetivos principales:
 - Automatizar el ciclo de vida completo de ML
 - Acelerar el despliegue de modelos a producción
 - Garantizar reproducibilidad y trazabilidad
 - Monitorear performance y detectar degradación
 - Facilitar colaboración entre equipos
- Componentes más importantes:
 - CI/CD para ML (entrenamiento y despliegue automatizado)
 - Versionado de datos, código y modelos
 - Monitoreo continuo y detección de drift
 - Governance y compliance
 - Infraestructura escalable

Introducción a AIOps

- AIOps (***Artificial Intelligence for IT Operations***) aplica ML y Big Data analytics a operaciones de TI para automatizar y mejorar la administración de infraestructura y aplicaciones.
- Diferencia importante: Mientras MLOps administra modelos de ML, AIOps usa ML para administrar operaciones de TI.
- Casos de uso:
 - Detección proactiva de anomalías en sistemas
 - Predicción de fallos de infraestructura
 - Automatización de respuesta a incidentes
 - Análisis de causa raíz automatizado
 - Optimización de recursos y capacidad
- En proyectos SCRUM: AIOps puede ser el producto que el equipo desarrolla, o herramientas que mejoran la operación del propio equipo.

Introducción a LLMOps

- LLMOps (***Large Language Model Operations***) es la evolución de MLOps específica para modelos de lenguaje grandes como GPT, Claude, Llama, etc.
- Desafíos súper específicos de LLMs:
 - Tamaño masivo de modelos (billones de parámetros)
 - Costos computacionales significativos
 - Prompt engineering como parte crítica del desarrollo
 - Riesgos de alucinaciones y outputs inapropiados
 - Necesidad de guardrails y moderación de contenido
- Componentes específicos:
 - Administración de prompts y templates
 - Vector databases para RAG (Retrieval Augmented Generation)
 - Evaluación cualitativa y cuantitativa de respuestas
 - Fine-tuning y adaptación de modelos
 - Administración de costos de API calls

Introducción a GenAIOps

- GenAIOps (**Generative AI Operations**) amplía el concepto más allá de LLMs para incluir cualquier modelo generativo: texto, imagen, audio, video, código.
- Alcance de GenAIOps:
 - Texto: LLMs, chatbots, generación de contenido
 - Imagen: DALL-E, Midjourney, Stable Diffusion
 - Audio: Generación de voz, música
 - Video: Generación y edición de video
 - Código: GitHub Copilot, Code Llama
- Prácticas más importantes:
 - Evaluación de calidad de outputs generativos
 - Administración de costos y cuotas de APIs
 - Seguridad y filtrado de contenido
 - Versionado de prompts y configuraciones
 - A/B testing de diferentes modelos/prompts
 - Cumplimiento legal y derechos de autor



Introducción a las Finanzas y a la Empresa

Eduardo Selim Martínez Mayorga

2026-1

Licenciatura en
Ciencia de Datos



Sesión: 28 de abril de 2026

*Es una
de las ceremonias
SCRUM*

Todos los proyectos se dividen en **sprints** → periodos de trabajo de 2-3 semanas

Sprint planning

Siempre terminan en un **entregable**

↳ planificación de las actividades de un sprint.

- El **sprint planning** inicia el sprint estableciendo el trabajo a realizar.
- Es un evento time-boxed de máximo 8 horas para un sprint de un mes
- Las 2 preguntas más importantes:
- **¿Qué puede entregarse en el incremento resultante del Sprint?**
 - El equipo de desarrollo pronostica la funcionalidad a desarrollar
 - Se define el objetivo del sprint
- **¿Cómo se logrará el trabajo necesario?**
 - El equipo de desarrollo descompone elementos en tareas de un día o menos
 - Se crea el sprint backlog
- En DS se seleccionan experimentos, análisis o componentes de modelo a desarrollar y validar

Product owner
Scrum master
Work technicians

¿Qué vamos a hacer?

¿Cómo lo vamos a hacer?

Mientras más detallado mejor

→ En DS no se sabe con certeza el resultado.

Otra de las ceremonias.

Daily SCRUM - Sincronización diaria

- El **daily SCRUM** es un evento de 15 minutos para el equipo de desarrollo que se realiza cada día del sprint.
- Busca sincronizar actividades y crear un plan para las próximas 24 horas.
- Estructura típica (tres preguntas):
 - ¿Qué hice ayer que ayudó al equipo a cumplir el objetivo del sprint?
 - ¿Qué haré hoy para ayudar al equipo a cumplir el objetivo del sprint?
 - ¿Veo algún impedimento que evite que el equipo o yo cumplamos el objetivo?
- **Beneficios:** Mejora la comunicación, elimina otras reuniones, identifica impedimentos, promueve la toma rápida de decisiones y mejora el conocimiento del equipo.

→ Formata pedir ayuda.

Lalo: El regalo del desarrollo de software al mundo

Sprint review

Otra
ceremonia.

¿qué
entregamos?
¿Está alineado
sufic. con el
product
final?

- Se realiza **al final del sprint** para inspeccionar el “incremento” y adaptar el *product backlog* en caso de ser necesario
- Es una reunión informal de máximo 4 horas para un sprint de un mes
- Actividades indispensables:
 - El product owner identifica qué se ha completado y qué no
 - El equipo de desarrollo demuestra el trabajo "hecho"
 - Se discute qué salió bien, problemas encontrados y cómo se resolvieron
 - Se revisa el product backlog y fechas probables de entrega

Asincrona

En DS se presentan resultados de experimentos, métricas de modelos, insights descubiertos y se valida el valor de negocio.

Sprint retrospective

→ otra ceremonia.
(interna)

- Es una oportunidad para que el equipo/célula SCRUM se inspeccione a sí mismo y cree un plan de mejoras para el próximo sprint
- Ocurre después del sprint review y antes del siguiente sprint planning.
- **Objetivo:** Inspeccionar cómo fue el último sprint en cuanto a personas, relaciones, procesos y herramientas.
- **Resultado:** Plan de acción con mejoras implementables en el próximo Sprint.
- Punto de partida - Contestar al menos las siguientes preguntas:
 - ¿Qué salió bien?
 - ¿Qué podría mejorar?
 - ¿Qué compromisos de mejora haremos?

En DS se suele revisar proceso de experimentación, calidad de datos, colaboración técnica.

 $\bar{0}$ delta's, Δ 's

“Incremento” a.k.a producto entregable

- Un “incremento” es la suma de todos los elementos del product backlog completados durante un sprint y el valor de los incrementos de todos los sprints anteriores
- Requisitos básicos:
 - Debe estar en condición "utilizable" sin importar si el product owner decide liberarlo
 - Debe cumplir la "definición de hecho" del equipo
 - Es un paso hacia una visión o meta
 - Debe funcionar independientemente de que se decida liberarlo

En DS puede ser un modelo entrenado, pipeline automatizado, dashboard, análisis documentado o experimento reproducible

Definición de hecho (*definition of done*)

- Es un entendimiento compartido de lo que significa que el trabajo esté completo
- Cuando un elemento del product backlog cumple la **definición de hecho**, nace un “incremento”
- Ejemplo para proyectos tradicionales:
 - Código escrito y revisado
 - Pruebas unitarias pasando
 - Documentación actualizada
 - Aceptado por el product owner
- Ejemplo para DS:
 - Código versionado en repositorio
 - Datos y experimentos reproducibles (MLflow, DVC [*Data Version Control*])
 - Modelo evaluado con métricas definidas
 - Notebook/documento explicando resultados
 - Validado con stakeholders

Adaptación de SCRUM a DS - Sprints exploratorios

- En DS, los primeros sprints suelen ser más exploratorios y orientados a entender los datos y validar la viabilidad del proyecto.
- **Sprint 0-1: Exploración y viabilidad**
 - Análisis exploratorio de datos (EDA)
 - Definición de métricas de éxito
 - Validación de calidad y disponibilidad de datos
 - Prototipo rápido (baseline model)
 - Estimación de viabilidad técnica
- **Entregable:** Informe de viabilidad, baseline model, comprensión del dominio
- **Objetivo:** Decidir si continuar, pivotar o cancelar el proyecto antes de invertir más recursos

Adaptación de SCRUM a DS - *User stories*

- Las **user stories** tradicionales ("Como [usuario] quiero [funcionalidad] para [beneficio]") deben adaptarse para DS
- Ejemplos adaptados:
 - **Análisis exploratorio:** "Como product owner necesito entender la distribución de churn en clientes para identificar segmentos críticos"
 - **Modelo predictivo:** "Como gerente de marketing necesito predecir la probabilidad de churn con 80% de precisión para optimizar retención"
 - **Pipeline de datos:** "Como científicx de datos necesito un pipeline automatizado de ETL para reducir el tiempo de preparación de datos"

Adaptación de SCRUM a DS - *Definition of done*

- La *definition of done* se debe adaptar a la naturaleza experimental de DS
- Criterios específicos en DS:
 - **Reproducibilidad:** Experimentos documentados con semillas aleatorias, versiones de librerías
 - **Versionado:** Datos, código y modelos trackeados (Git, DVC, MLflow)
 - **Evaluación:** Métricas calculadas en conjunto de validación/test
 - **Explicabilidad:** Interpretación de resultados y features importantes
 - **Revisión de código:** Peer review de notebooks y scripts
 - **Documentación:** README, comentarios, arquitectura del modelo
 - **Validación de negocio:** Métricas alineadas con objetivos de negocio

Product backlog en DS

- El product backlog en proyectos de DS incluye elementos técnicos y de negocio específicos del ciclo de vida de machine learning
- Tipos de elementos:
 - **Exploración:** Análisis de fuentes de datos, feature engineering, visualizaciones
 - **Experimentación:** Probar algoritmos, optimización de hiperparámetros, validación cruzada
 - **Infraestructura:** Pipelines de datos, automatización, monitoreo de modelos
 - **Modelado:** Desarrollo, entrenamiento y evaluación de modelos
 - **Deployment:** Containerización, APIs, integración con sistemas
 - **Monitoreo:** Detección de drift, performance tracking, alertas

Roles adicionales de DS en SCRUM

- Además de los roles clásicos de SCRUM, los proyectos de DS pueden requerir roles especializados dentro del equipo de desarrollo.
- **Data engineer:** Responsable de pipelines de datos, infraestructura, ETL y gestión de bases de datos.
- **Data scientist:** Desarrolla modelos, realiza análisis estadísticos y experimentos.
- **ML Engineer:** Especialista en productivización, deployment y escalabilidad de modelos.
- **Domain expert:** Aporta conocimiento del negocio y valida hipótesis (puede ser el product owner o colaborador cercano).

Nota importante: Todxs siguen siendo parte del equipo de desarrollo, sin jerarquías, colaborando para el objetivo común.

Métricas y monitoreo en SCRUM para DS

- Las métricas en proyectos DS con SCRUM deben cubrir tanto el proceso ágil como el rendimiento técnico
- Métricas de proceso:
 - Velocity (puntos completados por sprint)
 - Burndown charts (progreso del sprint)
 - Cycle time (tiempo desde inicio hasta completado)
- Métricas de producto (específicas de DS):
 - Métricas de modelo: accuracy, precision, recall, F1, AUC-ROC
 - Métricas de negocio: ROI, reducción de costos, aumento de conversión
 - Calidad de datos: completitud, consistencia, actualidad
 - Model drift: degradación del performance en producción
 - Reproducibilidad: % de experimentos reproducibles

RMSE
AUC, R²
BK

Administración de experimentos vía sprints

- La experimentación es importante en DS y debe integrarse naturalmente en el flujo de SCRUM
- Mejores prácticas:
 - a. **Hipótesis claras:** Cada experimento debe tener una hipótesis testeable definida en el Sprint Planning
 - b. **Tracking sistemático:** Usar herramientas como MLflow, Weights & Biases o Neptune.ai para registrar experimentos
 - c. **Criterios de éxito:** Definir métricas y umbrales antes de ejecutar experimentos
 - d. **Time-boxing:** Limitar tiempo de experimentación para evitar "rabbit holes"
 - e. **Compartir resultados:** Experimentos negativos también son valiosos - documentar lecciones aprendidas

Manejo de la deuda técnica en DS

¿Qué es deuda técnica en desarrollo de software?

Gemini: Costo x retrabajo futuro.

causado por decisión rápida y fácil hoy

ChatGPT: Costo x decisiones de desarrollo hoy.

Claude: Acumulación de decisiones de desarrollo

- La deuda técnica en DS incluye:
 - Notebooks desordenados
 - Código duplicado
 - Falta de tests
 - Modelos sin documentación
- SCRUM ayuda a administrar dicha deuda técnica sistemáticamente
- Estrategias:
 - **Incluir en el backlog:** Refactorización y mejoras técnicas deben ser items prioritizados
 - **Regla del 20%:** Dedicar ~20% de cada sprint a reducir deuda técnica
 - **Definition of done muy estricta:** Prevenir nueva deuda exigiendo calidad desde el inicio
 - **Retrospectivas:** Identificar patrones que generan deuda técnica
 - **Pair programming:** Revisión de código entre científicos de datos para mantener estándares

Beneficios y retos de SCRUM para DS

Beneficios:

- **Transparencia:** Stakeholders ven progreso y resultados cada sprint
- **Adaptabilidad:** Pivotar rápido según hallazgos en los datos
- **Entrega de valor:** Modelos incrementales vs. "big bang" al final
- **Reducción de riesgo:** Una validación temprana evita meses de trabajo inútil
- **Colaboración:** Equipos multidisciplinarios trabajando sincronizados
- **Mejora continua:** Retrospectivas optimizan proceso constantemente

Retos:

- La incertidumbre intrínseca de DS dificulta estimaciones precisas
- Hacer investigación puede no caber en sprints fijos
- El equilibrio entre exploración y entrega no es trivial
- Resistencia a "trabajar sin saber la respuesta"
- Requiere madurez del equipo y organización

Herramientas para SCRUM en DS

Integrar herramientas de administración ágil con herramientas de DS facilita el flujo de trabajo.

Administración de proyectos:

- Jira, Azure DevOps, Trello: Product/Sprint Backlog
- Confluence, Notion: Documentación
- Slack, Teams: Comunicación diaria

Desarrollo de DS:

- Git/GitHub/GitLab: Versionado de código
- DVC: Versionado de datos y modelos
- MLflow, Weights & Biases: Tracking de experimentos
- Docker: Reproducibilidad de entornos
- Jupyter, VSCode: Desarrollo interactivo

Introducción a MLOps

Development
opr.

- MLOps (**Machine Learning Operations**) es un conjunto de prácticas que combina **Machine Learning**, **DevOps** y **Data Engineering** para implementar, monitorear y mantener modelos de ML en producción de manera confiable y eficiente.
- Objetivos principales:
 - Automatizar el ciclo de vida completo de ML
 - Acelerar el despliegue de modelos a producción
 - Garantizar reproducibilidad y trazabilidad
 - Monitorear performance y detectar degradación
 - Facilitar colaboración entre equipos
- Componentes más importantes:
 - CI/CD para ML (entrenamiento y despliegue automatizado)
 - Versionado de datos, código y modelos
 - Monitoreo continuo y detección de drift
 - Governance y compliance
 - Infraestructura escalable

degradación
del modelo

MLOps en el contexto de SCRUM

- **MLOps y SCRUM se complementan bien:** SCRUM organiza el trabajo iterativo, MLOps automatiza y operacionaliza los entregables.
- Integración en sprints
- **Sprint planning:** Incluir tareas de automatización, testing y deployment en el sprint backlog
- **Durante el sprint:**
 - Desarrollo de features → Commits → CI/CD automático
 - Experimentos trackeados automáticamente
 - Tests automatizados de modelos
- **Sprint review:** Demostrar modelos desplegados en ambientes de staging/producción
- **Retrospectiva:** Evaluar eficiencia de pipelines y procesos MLOps
- **Definition of done extendida:** Un modelo está "hecho" cuando está desplegado, monitoreado y documentado.

Introducción a AIOps

- AIOps (***Artificial Intelligence for IT Operations***) aplica ML y Big Data analytics a operaciones de TI para automatizar y mejorar la administración de infraestructura y aplicaciones.
- Diferencia importante: Mientras MLOps administra modelos de ML, AIOps usa ML para administrar operaciones de TI.
- Casos de uso:
 - Detección proactiva de anomalías en sistemas
 - Predicción de fallos de infraestructura
 - Automatización de respuesta a incidentes
 - Análisis de causa raíz automatizado
 - Optimización de recursos y capacidad
- En proyectos SCRUM: AIOps puede ser el producto que el equipo desarrolla, o herramientas que mejoran la operación del propio equipo.

AIOps & MLOps - Diferencias y complementariedad

Aunque relacionados, AIOps y MLOps tienen enfoques y objetivos distintos.

MLOps

- Objetivo: Operacionalizar modelos de ML
- Usuarios: Data Scientists, ML Engineers
- Scope: Ciclo de vida de modelos de ML
- Métricas: Accuracy, precision, latencia de inferencia

AIOps

- Objetivo: Automatizar operaciones de TI con IA
- Usuarios: SREs, DevOps, IT Operations
- Scope: Infraestructura, aplicaciones, redes
- Métricas: MTTR, uptime, resolución automática de incidentes

Complementariedad: MLOps despliega modelos que AIOps usa para monitorear sistemas; AIOps optimiza la infraestructura donde corren los pipelines de MLOps.

Introducción a LLMOps

- LLMOps (***Large Language Model Operations***) es la evolución de MLOps específica para modelos de lenguaje grandes como GPT, Claude, Llama, etc.
- Desafíos súper específicos de LLMs:
 - Tamaño masivo de modelos (billones de parámetros)
 - Costos computacionales significativos
 - Prompt engineering como parte crítica del desarrollo
 - Riesgos de alucinaciones y outputs inapropiados
 - Necesidad de guardrails y moderación de contenido
- Componentes específicos:
 - Administración de prompts y templates
 - Vector databases para RAG (Retrieval Augmented Generation)
 - Evaluación cualitativa y cuantitativa de respuestas
 - Fine-tuning y adaptación de modelos
 - Administración de costos de API calls

Introducción a GenAIOps

- GenAIOps (**Generative AI Operations**) amplía el concepto más allá de LLMs para incluir cualquier modelo generativo: texto, imagen, audio, video, código.
- Alcance de GenAIOps:
 - Texto: LLMs, chatbots, generación de contenido
 - Imagen: DALL-E, Midjourney, Stable Diffusion
 - Audio: Generación de voz, música
 - Video: Generación y edición de video
 - Código: GitHub Copilot, Code Llama
- Prácticas más importantes:
 - Evaluación de calidad de outputs generativos
 - Administración de costos y cuotas de APIs
 - Seguridad y filtrado de contenido
 - Versionado de prompts y configuraciones
 - A/B testing de diferentes modelos/prompts
 - Cumplimiento legal y derechos de autor

→ Universo Claude (2026)

Introducción a DataOps

- DataOps aplica principios ágiles y DevOps a la administración de datos
- Indispensable para cualquier iniciativa de MLOps/LLMOps
- Principios de DataOps:
 - Automatización de pipelines de datos
 - Validación continua de calidad de datos
 - Versionado de datasets
 - Monitoreo de data lineage
 - Colaboración entre data engineers y consumers
- Relación con SCRUM:
 - Product backlog: incluye mejoras de calidad de datos
 - Sprint goal: puede enfocarse en pipeline crítico de datos
 - Definition of done: datos validados, documentados, versionados

El espectro de XOps en proyectos vía SCRUM

Entender cómo los diferentes XOps se relacionan ayudan a estructurar el product backlog y sprints.

Capas operacionales:

1. DataOps (Base): Pipelines de datos de calidad
2. MLOps (Modelos tradicionales): Clasificación, regresión, clustering
3. LLMOps (Modelos de lenguaje): Chatbots, análisis de texto, generación
4. GenAIOps (Multimodal): Texto + imagen + audio + video
5. AIOps (Consumidor): Usa outputs de modelos para operaciones

En cada sprint el equipo puede trabajar en varias capas, pero la base (DataOps) es pre-requisito para las superiores.

Métricas XOps para sprint reviews

- Las métricas XOps deben demostrarse en los sprint reviews para evidenciar progreso tangible
- **MLOps:**
 - Deployment frequency (despliegues por semana)
 - Lead time (tiempo de experimento a producción)
 - Mean time to recovery (MTTR) de modelos
 - Model prediction latency (p50, p95, p99)
 - % de modelos con drift detectado
- **LLMOps:**
 - Costo por 1K tokens/requests
 - Latencia de generación (time to first token)
 - Tasa de uso de caché
 - % de respuestas filtradas por guardrails
 - Satisfacción de usuario (thumbs up/down)

Métricas XOps para sprint reviews

DataOps:

- Data quality score (completitud, validez)
- Pipeline execution time
- Data freshness (lag desde fuente)
- Schema violations detectadas

Roles XOPs en la célula SCRUM

- Los equipos SCRUM para proyectos XOps requieren habilidades especializadas pero mantienen la colaboración multifuncional
- Roles especializados dentro del development team:
 - **MLOps Engineer:** Automatiza pipelines ML, gestiona infraestructura, implementa CI/CD
 - **Data Engineer:** Construye pipelines de datos, asegura calidad, gestiona almacenamiento
 - **ML/AI Engineer:** Desarrolla modelos, experimenta con algoritmos, optimiza performance
 - **LLM Engineer:** Especialista en prompt engineering, RAG, fine-tuning de LLMs
 - **SRE/Platform Engineer:** Administra Kubernetes, monitoreo, escalabilidad

Importante: En SCRUM, estos son especializaciones dentro del equipo, no silos. Todxs colaboran en el sprint goal común y se apoyan entre sí.

Deuda técnica en entorno XOps

Los proyectos XOps acumulan deuda técnica específica que debe gestionarse en el Product Backlog.

- MLOps:
 - Modelos en producción sin reentrenamiento automático
 - Ausencia de tests de modelos (unit, integration)
 - Falta de monitoreo de drift
 - Pipelines frágiles con dependencias hardcodeadas
- LLMOps:
 - Prompts hardcodeados sin versionado
 - Sin fallbacks cuando API falla
 - Ausencia de evaluaciones sistemáticas
 - Costos descontrolados sin caching
- DataOps:
 - Pipelines sin validación de calidad
 - Data lineage no documentado
 - Transformaciones no reproducibles

Estrategia SCRUM: Dedicar items del sprint backlog explícitamente a reducir esta deuda.

CI/CD para XOps - Pipelines automatizados

- La automatización CI/CD es crítica para operacionalizar modelos y mantener agilidad en sprints.
- **Pipeline CI** para ML:
 - Trigger: Push a repo o schedule
 - Validación de datos (great expectations :()
 - Entrenamiento de modelo
 - Evaluación con thresholds mínimos
 - Tests unitarios de código ML
 - Si pasa: registrar modelo en Model Registry
- **Pipeline CD** para ML:
 - Trigger: Nuevo modelo aprobado
 - Deployment a staging
 - Smoke tests y validación
 - A/B testing o canary deployment
 - Monitoreo automático activado
 - Rollback automático si métricas degradadas

En un sprint cada mejora al pipeline es un item entregable del backlog

Feature stores en MLOps

Los Feature stores son infraestructura que centraliza, versiona y sirve features para entrenamiento e inferencia.

- Beneficios:
 - Reutilización: Features creadas una vez, usadas por varios modelos
 - Consistencia: Mismas features en training y serving
 - Velocidad: Reducen time-to-market de nuevos modelos
 - Gobierno: Control de acceso y linaje de features
- En SCRUM:
 - Crear feature store puede ser objetivo de varios sprints
 - Migrar modelos a usar feature store es trabajo incremental
 - El Product Owner prioriza features según valor de negocio

Monitoreo y observabilidad en producción

El monitoreo continuo sirve para detectar problemas antes de que impacten al negocio
¿Qué monitorear?

- Métricas de Modelo:
 - Performance: accuracy, precision, recall en ventanas temporales
 - Data drift: cambios en distribución de inputs
 - Concept drift: cambios en relación input-output
 - Prediction drift: cambios en distribución de predicciones
- Métricas de Sistema:
 - Latencia (p50, p95, p99)
 - Throughput (predicciones/segundo)
 - Error rate (% de requests fallidas)
 - Utilización de recursos (CPU, RAM, GPU)
- Métricas de Negocio:
 - Impacto en KPIs (conversión, revenue)
 - Satisfacción de usuario
- En el sprint review idealmente se debe mostrar dashboards de monitoreo funcionando.

Manejo de experimentos en entornos XOps

La experimentación sistemática requiere infraestructura y proceso, items recurrentes en el product backlog.

Mejores prácticas:

- **1. Tracking centralizado:**
 - Todos los experimentos registrados en MLflow/W&B
 - Metadata: hiperparámetros, métricas, artefactos
 - Código versionado con hash de commit
- **2. Nomenclatura consistente:**
 - Convención de nombres para experimentos
 - Tags para filtrar (modelo-tipo, dataset-version, autor)
- **3. Comparación de experimentos:**
 - Dashboards para comparar runs
 - Selección automática del mejor modelo
- **4. Reproducibilidad:**
 - Environment pinning (requirements.txt, Docker)
 - Random seeds fijadas
 - Datasets versionados con DVC
- Sprint goal ejemplo: "100% de experimentos reproducibles con un comando"

Un centavo de seguridad y compliance en XOps

- La seguridad y cumplimiento regulatorio son responsabilidades compartidas del equipo SCRUM.
- **MLOps:**
 - Modelos pueden ser atacados (adversarial attacks)
 - Data poisoning en entrenamiento
 - Model stealing a través de APIs
 - Privacidad: PII en datasets de entrenamiento
- **LLMOps:**
 - Prompt injection attacks
 - Fuga de información sensible en respuestas
 - Jailbreaking de guardrails
 - Contenido tóxico o sesgado
- **En el definition of done se debe incluir una checklist de seguridad y compliance**

Estrategias para deployment de modelos

- La forma de desplegar modelos impacta el riesgo y la velocidad, decisiones que toma el equipo SCRUM.
- **Shadow deployment:**
 - Modelo nuevo recibe tráfico pero no afecta decisiones
 - Se comparan predicciones con modelo actual
 - Cuándo: Validar modelo en datos reales sin riesgo
- **Canary deployment:**
 - Modelo nuevo recibe pequeño % de tráfico (5-10%)
 - Monitoreo intensivo, rollback si problemas
 - Cuándo: Introducir modelo gradualmente

Estrategias para deployment de modelos

- **A/B testing:**
 - Usuarios divididos aleatoriamente entre modelos
 - Comparación estadística de métricas de negocio
 - Cuándo: Medir impacto en KPIs antes de full rollout
- **Blue-Green deployment:**
 - Dos ambientes: producción (blue) y nuevo (green)
 - Switch instantáneo de todo el tráfico
 - Cuándo: Rollback rápido es crítico

Cultura DevOps/MLOps en equipos SCRUM

La transformación XOps requiere cambio cultural que SCRUM puede facilitar.

Principios culturales:

1. **You build it, you run it:** El equipo que desarrolla el modelo lo opera en producción
2. **Fail fast, learn faster:** Experimentación rápida con feedback loops cortos
3. **Automation over heroics:** Evitar dependencia de individuos "heroicos" que arreglan todo manualmente
4. **Blameless postmortems:** Cuando falla un modelo, aprender sin culpar
5. **Shared responsibility:** Todo el equipo es responsable de la salud del sistema

Cómo SCRUM reforza esto:

- Daily scrum: Comunicación transparente sobre problemas
- Retrospectives: Mejora continua de procesos
- Definition of done: Estándares de calidad compartidos

Antipatrones a evitar en XOps con SCRUM

Reconocer antipatrones ayuda al equipo a evitar trampas comunes.

- Antipatrones MLOps:
 - **"Notebook Hell"**: Notebooks desordenados sin código productizable
 - **"Model Zoo"**: Muchos modelos experimentales, ninguno en producción
 - **"Works on my machine"**: Falta de reproducibilidad
 - **"Fire and forget"**: Desplegar modelo sin monitoreo
- Antipatrones LLMOps:
 - **"Prompt spaghetti"**: Prompts hardcodeados por todo el código
 - **"Cost blindness"**: Sin control de gastos en APIs
 - **"Evaluation by vibe"**: Sin métricas objetivas de calidad
- Antipatrones SCRUM:
 - **"Waterfall in sprints"**: Análisis completo → Desarrollo completo → Testing
 - **"No definition of done"**: Sin criterios claros de completitud



Introducción a las Finanzas y a la Empresa

Eduardo Selim Martínez Mayorga

2026-2

Licenciatura en
Ciencia de Datos



Sesión: 5 de mayo de 2026
7

¿Qué es el Trading?

- **Definición:** Proceso de compra y venta de activos financieros con el objetivo de obtener beneficios
 - Diferencia con inversión:
 - Trading: corto plazo → aprovechar los movimientos de precios
 - Inversión: largo plazo → basado en valor fundamental
 - Enfoques: Discrecional vs. Sistemático
- Handwritten notes:*
→ busca ineficiencias de oportunidades de arbitraje
(circled around "aprovechar los movimientos de precios")

Tipos principales de trading

- **Day trading:** Operaciones que se abren y cierran el mismo día
- **Swing trading:** Operaciones de días a semanas
- **Position trading:** Operaciones de semanas a meses
- **Scalping:** Operaciones muy rápidas (minutos)
- **High-frequency trading:** Usa algoritmos complejos y sistemas ultrarrápidos para ejecutar un gran volumen de órdenes en fracciones de segundo

¿Qué activos se tradean?

- Renta variable: Acciones
- Renta fija: Bonos, instrumentos de deuda
- Derivados: Futuros, opciones, swaps, etc.
- Commodities: Petróleo, oro, agrícolas, etc.
- Divisas (Forex): EUR/USD, MXN/USD, etc.
- Criptoactivos: Bitcoin, Ethereum, stablecoins, etc.

Datos de mercado - OHLCV

- Los datos básicos del trading se representan en el formato estándar OHLCV:
 - **Open:** Precio de apertura del período
 - **High:** Precio máximo alcanzado
 - **Low:** Precio mínimo alcanzado
 - **Close:** Precio de cierre del período
 - **Volume:** Volumen de operaciones
- Los timeframes definen la granularidad temporal de los datos:
 - 1m, 5m, 15m: Trading intradía
 - 1h, 4h: Swing trading
 - 1D: Trading diario
 - 1W, 1M: Análisis de largo plazo
- El timeframe correcto depende de la estrategia y estilo de trading

Se empieza sencillo... con Análisis Técnico

- **Análisis técnico:** Estudio de patrones históricos de precios y volúmenes para predecir movimientos futuros.
- Premisas básicas:
 - El precio engloba toda la información del activo en consideración
 - Los precios se mueven en tendencias
 - La historia tiende a repetirse
- El análisis técnico es ideal para aplicar machine learning

Los modelos
neúles surgen
con
caracteres
de uso

Modelos de reconocimiento
de patrones, series de tiempo

¿Quiénes participan en esto del trading?

- **Market makers:** También se conocen como proveedores de liquidez
 - Publican simultáneamente un precio de compra (bid) y otro de venta (ask)
 - Es decir dos precios → uno al que está dispuesto a comprar (bid) y otro al que vender (ask)
 - La ganancia la obtienen de la diferencia entre estos dos precios, i.e. el spread bid-ask.
- **Institucionales:** Fondos de inversión, hedge funds, bancos.
- **Retail traders:** Individuos
- **Arbitrageurs**
- **High-frequency traders**
- Reguladores y bolsas

Fulanos que están
buscando oportunidades
de arbitraje
no gana \$ sin
neto \$.

Libro de órdenes (order book) a.k.a microestructura de mercado

- El **order book** es un registro dinámico en tiempo real de todas las órdenes de compra (bids) y venta (asks) activas en el mercado.
- Se organiza por niveles de precios y cantidades, mostrando la profundidad de mercado (market depth).
- Es fundamental para analizar la liquidez disponible en cada precio y anticipar movimientos de corto plazo.
- No todas las órdenes son lo mismo:
 - Nivel 1: Mejor precio bid y ask (top of the book).
 - Nivel 2: Profundidad adicional (otras órdenes pendientes más allá del mejor precio).

Tipos de órdenes

- **Orden de mercado (market order):** Ejecuta inmediatamente al mejor precio disponible → rápida, pero vulnerable a slippage.
- **Orden límite (limit order):** Establece un precio máximo (compra) o mínimo (venta); sólo se ejecuta si el mercado alcanza ese precio → más control, menos garantía de ejecución.
- **Stop-loss:** Orden que se activa cuando el precio cruza un umbral → usada para limitar pérdidas automáticamente.
- **Stop-limit:** Combinación de stop y límite.
- **Iceberg:** Sólo una parte visible en el libro, para ocultar tamaño real.

Tipos de órdenes

- Los diferentes tipos de órdenes afectan la dinámica del order book y generan diferentes patrones en los datos de alta frecuencia.
- Detectar el flujo de órdenes “pasivas” v.s. “agresivas” puede alimentar modelos predictivos de microestructura.

Matching engine & ejecución

- Se refiere a cómo se matchean/corresponden las órdenes en el mercado
- El matching engine es el núcleo tecnológico de un mercado electrónico → matchean órdenes según reglas predefinidas.
- Principios comunes:
 - Price priority: El mejor precio se ejecuta primero.
 - Time priority: De entre órdenes al mismo precio, la más antigua se ejecuta antes.
- Orden agresiva → toma liquidez (market order)
- Orden pasiva → provee liquidez (limit order).
- Para estrategias algorítmicas, entender la cola de órdenes y el tiempo de permanencia es casi que lo más importante.
- Latencias milisegundo/nanosegundo son determinantes en mercados electrónicos (e.g., HFT).

Algunos tipos de estrategias cuantitativas

- **Trend-following:** Explotan el momentum → SMA crossovers, breakout systems.
- **Mean-reversion:** Aprovechan la reversión a la media → pair trading, Bollinger band.
- **Market-making:** Provee liquidez capturando spread → HFT.
- **Arbitrage:** Busca ineficiencias → estadístico, cross-asset, triangular.
- **Event-driven:** Explota reacciones a eventos → earnings, noticias, políticas.
- DS ayuda a detectar patrones, modelar señales y evaluar desempeño de cada tipo.

Indicadores técnicos comunes

- **Indicadores técnicos:** señales simples sobre datos crudos
- Los indicadores técnicos buscan resumir patrones de precio y volumen
- Algunos populares:
 - **Media móvil simple (SMA)**
 - **Media móvil exponencial (EMA)**
 - **RSI (Relative Strength Index):** mide momentum → sobrecompra/sobreventa.
 - **MACD (Moving Average Convergence Divergence):** diferencia entre dos medias móviles exponenciales → cruces de señal
 - **Bollinger Bands:** bandas = $SMA \pm k \cdot \sigma$ → volatilidad dinámica
 - **ADX (Average Directional Index)**
 - **Stochastic Oscillator**
 - **CCI (Commodity Channel Index)**

DS puede tratarlos como variables explicativas en modelos supervisados.

Backtesting

- Proceso de probar una estrategia con datos históricos
- Algorítmica clásica:
 - Definir reglas de entrada y salida
 - Aplicar la estrategia a datos pasados
 - Calcular métricas de performance
 - Optimizar parámetros
 - Validar con datos out-of-sample
- Cuidado con el overfitting → el modelo debe generalizar a datos nuevos.

Métricas de evaluación

- Específicas de trading:
 - Sharpe Ratio
 - Maximum Drawdown (caída máxima)
 - Win Rate (% operaciones ganadoras)
 - Profit Factor (ganancias/pérdidas)
 - Calmar Ratio
- Las clásicas de ML:
 - Precision, Recall, F1-Score
 - ROC-AUC
 - Matriz de confusión

Análisis walk-forward

- Método robusto de validación en el tiempo:
 - Entrenar modelo con ventana de datos
 - Predecir en período siguiente
 - Mover la ventana hacia adelante
 - Repetir el proceso
- Evita el look-ahead bias (usar información futura)
- Valida la habilidad predictiva real del modelo.

Diseño de estrategias de trading y ML

- A primera vista, parecen disciplinas distintas:
 - Trading → decisiones de compra/venta
 - ML supervisado → predicción de etiquetas o valores
- Pero comparten una misma estructura lógica:
 - Input: Datos de mercado / features
 - Transformación: Algoritmo / señal / modelo
 - Output: Señal de acción (trade) o predicción
 - Evaluación: Backtest o métricas de error
- En el trading cuantitativo moderno, una estrategia es un modelo predictivo con un paso de ejecución y administración de riesgo

Diseño de estrategias de trading & ML

- Estructura de datos:
 - Ambos utilizan features (lags, indicadores, embeddings...)
- Objetivo predictivo:
 - ML: Minimizar error de predicción
 - Trading: Anticipar dirección o magnitud para maximizar P&L
- Proceso de entrenamiento / calibración:
 - Ajuste de parámetros, selección de features, validación
- Validación temporal:
 - Walk-forward, train/test por periodos
- Riesgo de overfitting:
 - En ML: Sobreajuste a datos de entrenamiento
 - En trading: Sobreoptimización de parámetros históricos
- Importancia de la generalización:
 - Ambos deben funcionar fuera de muestra (out-of-sample)

Diseño de estrategias de trading & ML

Pero....

- Un modelo ML no es automáticamente una estrategia de trading rentable
- Una estrategia rentable puede incorporar ML, pero también necesita capas adicionales: ejecución, control de riesgo, robustez y monitoreo
- En ML el output suele ser pasivo (predicción)
- En trading el output es activo (ejecución en mercado)

Diseño de estrategias de trading & ML

- Acá usaremos Python porque ya tiene muchas librerías armadas:
- Datos:
 - yfinance, pandas_datareader
 - ccxt (criptomonedas)
- Análisis:
 - pandas, numpy
 - TA-lib (indicadores técnicos)
 - matplotlib, plotly
- *Backtesting:*
 - backtrader
 - zipline
 - vectorbt